

Concurrency Control Techniques

1. Binary Locks (the simplest, impractical baseline)

Each data item has a lock variable with only two states: **0** (unlocked) / **1** (locked).

```
lock_item(X):  if LOCK(X)=0 → set to 1, grant access; else → wait
unlock_item(X): set LOCK(X)=0, wake one waiting transaction
```

Rules: lock before access, unlock after finishing, no double-locking, no unlocking without ownership.

Why binary locks aren't used in practice: only **one** transaction can hold a lock on an item at any time — even for pure **reads**, which could safely happen concurrently. This unnecessarily kills concurrency for the extremely common case of multiple readers.

2. Shared/Exclusive (Read/Write) Locks

Fixes binary locking's core flaw by distinguishing **read** intent from **write** intent.

Lock type	Multiple holders allowed?	Blocks other reads?	Blocks other writes?
Shared (read)	Yes — many transactions can hold it simultaneously	No	Yes
Exclusive (write)	No — only one holder	Yes	Yes

```
read_lock(X):  unlocked → read-locked, No_of_reads=1
               already read-locked → increment No_of_reads
               write-locked → wait
write_lock(X): unlocked → write-locked
               locked (either mode) → wait
unlock(X):     write-locked → unlock, wake one waiter
               read-locked → decrement No_of_reads; if 0, unlock, wake one waiter
```

Lock conversion:

- **Upgrade** (read → write) — allowed only if this transaction is the **sole** current reader; otherwise it must wait.
- **Downgrade** (write → read) — always fine (relinquishing exclusivity is never dangerous).

3. Why Locking Alone Doesn't Guarantee Serializability

Using locks correctly is not enough — *when you release them matters just as much.*

Worked counterexample: $X=20, Y=30$. $T1$: read Y ; unlock Y ; write $X = X+Y$. $T2$: read X ; write $Y = Y+X$.

If $T1$ releases its lock on Y **immediately after reading it** (rather than holding it until $T1$ is completely done), $T2$ can sneak in and modify Y before $T1$ finishes using the value it read:

```
1. T1 reads Y=30
2. T1 unlocks Y ← too early!
3. T2 reads X=20
4. T2 writes Y = Y+X = 50
5. T1 writes X = X+Y = 20+30 = 50 (using the STALE Y=30 it read earlier)

Final: X=50, Y=50
```

Compare against **both** possible serial orders:

```
T1 then T2: X=50, Y=80
T2 then T1: X=70, Y=50
```

Neither matches $X=50, Y=50$ — this interleaved execution is **not serializable**, despite every individual lock/unlock operation being "valid" on its own. The problem: $T1$'s later write to X still *depends* on the value it read from Y — releasing Y 's lock before finishing that dependency chain let $T2$ invalidate the assumption $T1$'s later step was built on.

4. Two-Phase Locking (2PL) — the fix

Rule: every transaction is split into exactly two phases:

- **Growing (expanding) phase** — can **acquire** new locks; **cannot release any**.
- **Shrinking phase** — can **release** locks; **cannot acquire any new ones**.

Once a transaction releases its *first* lock, it may never acquire another — this is what "two-phase" refers to.

Applying 2PL to the counterexample above:

```
T1: read_lock(Y); read(Y); write_lock(X); write(X:=X+Y); unlock(Y); unlock(X)
T2: read_lock(X); read(X); write_lock(Y); write(Y:=Y+X); unlock(X); unlock(Y)
```

Now $T2$ **cannot** acquire `write_lock(Y)` until $T1$ releases it — which $T1$ only does *after* it has already finished its own write to X . The forced wait produces:

T1 fully executes: $X=50$, $Y=30$ (unchanged so far)
T1 releases both locks
T2 then executes: reads $X=50$, writes $Y = 30+50 = 80$
Final: $X=50$, $Y=80$ ← matches the serial order $T1 \rightarrow T2$ exactly.

Why this guarantees serializability: by never releasing a lock before acquiring everything it still needs, a 2PL transaction can never let another transaction interfere with a dependency it's still relying on — this structurally rules out exactly the kind of scenario in §3. (Proving this rigorously is beyond what's needed for interview purposes — but the mechanism, "hold everything until you're truly done," is the entire intuition.)

2PL's own cost: it can cause **deadlock** — precisely because transactions hold locks for longer (through their entire growing phase), increasing the chance two transactions each hold something the other now needs.

5. Deadlock

Two or more transactions are each waiting **indefinitely** for a resource held by another, forming a cycle — none can ever proceed.

T1: holds X, wants Y → waiting
T2: holds Y, wants X → waiting

Prevention protocols (avoid deadlock by construction, before it can happen)

Protocol	Idea	Trade-off
Conservative 2PL	Lock <i>everything</i> a transaction will ever need, before it starts; if even one lock is unavailable, wait and retry the whole batch	Deadlock-free; but needs to know all needed items upfront, and hurts concurrency
Ordering items	Assign a fixed global order to all data items; every transaction must acquire locks in that order	Deadlock-free (no cycle can form if everyone locks in the same order); hard to manage in dynamic databases
Wait-Die (timestamp-based)	Older transaction requesting an item held by a younger one: waits . Younger requesting from an older: aborts and restarts	Only older transactions ever wait → no cycle possible
Wound-Wait (timestamp-based)	Older transaction requesting an item held by a younger one: preempts/aborts the younger ("wounds" it). Younger requesting from an older: waits	Only younger transactions ever wait → no cycle possible
No Waiting (NW)	Can't get a lock? Abort and restart immediately, no waiting at all	Deadlock-free trivially; but causes many unnecessary aborts
Cautious Waiting (CW)	Wait only if the lock-holder is not itself currently blocked	Deadlock-free (no cycle can form); fewer wasted aborts than NW

Wait-Die vs. Wound-Wait — the pattern to remember: both use transaction age (via timestamp) to decide who backs off, and both are specifically designed so that **only one direction of the age relationship is ever allowed to wait** — this asymmetry is exactly what makes a wait-for cycle structurally impossible in either scheme. The difference is just *which* side backs off and *how* (Wait-Die: the requester dies; Wound-Wait: the requester wounds the holder).

Deadlock Detection (let it happen, then find and fix it)

- Maintain a **wait-for graph**: node = transaction, edge $T_i \rightarrow T_j$ = "T_i is waiting for an item locked by T_j."
- **A cycle in this graph = deadlock.**
- **Victim selection** — when a cycle is found, abort one (or more) transactions to break it; prefer aborting transactions that have done **less work** (fewer completed updates), to minimize wasted effort.

- **Practical trade-off:** checking for cycles after *every* lock request is expensive — real systems check periodically or based on wait duration instead.

Timeouts (a cheap approximation of detection)

If a transaction waits longer than some threshold, just **assume** deadlock and abort it. Doesn't actually guarantee a real deadlock exists — but very low overhead, which is often an acceptable trade for simplicity.

Starvation — a distinct but related problem

A transaction **never** gets to acquire a lock, indefinitely — **not** because of a cycle, but because of unfair scheduling (e.g., always being the one selected as the victim, or always losing out to a stream of other requests).

Fixes: first-come-first-served queuing, priority aging (increase priority the longer a transaction waits), and careful victim selection (don't keep punishing the same transaction repeatedly). Wait-Die and Wound-Wait both naturally avoid starvation, since a transaction's *fixed* timestamp only ever becomes more favorable relative to newer transactions over time — it can't be perpetually the "loser."

Interview soundbite: "Deadlock is a cycle of mutual waiting; starvation is one transaction being perpetually unlucky, with no cycle involved at all. Prevention protocols stop deadlock before it can occur, by restricting *how* transactions are allowed to wait; detection lets it happen and then breaks the cycle after the fact — a classic prevent-vs-detect-and-recover trade-off, the same shape you see in OS deadlock handling."

6. Timestamp Ordering & MVCC

Basic Timestamp Ordering

Every transaction gets `TS(T)` (its start time — older transaction = smaller timestamp). Each data item tracks `read_TS` (latest transaction to read it) and `write_TS` (transaction that wrote its current value). A read/write is only allowed if it doesn't violate the timestamp order (e.g., a transaction is rejected/aborted if it tries to write a value that a *later*-timestamped transaction has already read, since that would retroactively invalidate what that later transaction saw).

MVCC (Multiversion Concurrency Control)

Instead of keeping just **one** value per item, keep **multiple versions**, each tagged with its own `write_TS` / `read_TS`.

Benefit: a read that would otherwise **conflict** and be forced to abort under plain timestamp ordering can often instead be served an **older version** that's still consistent with its own timestamp — increasing concurrency at the cost of extra storage (and it's a natural fit for temporal/audit-history use cases anyway).

Reading X (by transaction T): find the version x_i with the largest $\text{write_TS}(x_i) \leq \text{TS}(T)$ (the most recent version that existed *as of* T's timestamp), then update $\text{read_TS}(x_i) = \max(\text{read_TS}(x_i), \text{TS}(T))$.

Writing X (by transaction T): find the version x_i with the largest $\text{write_TS}(x_i) \leq \text{TS}(T)$. If $\text{read_TS}(x_i) > \text{TS}(T)$, **abort T** (a *later*-timestamped transaction already read this version — T's write would retroactively invalidate what it saw). Otherwise, create a **new version** x_j with $\text{write_TS}(x_j) = \text{read_TS}(x_j) = \text{TS}(T)$.

Multiversion 2PL (MV2PL) — combining locking with versions

- **Two versions per item:** a **committed** version (visible to readers) and an **uncommitted** version (private to the current writer).
- **Three lock modes:** Read (shared, on the committed version), Write (exclusive, for creating a new uncommitted version), **Certify** (exclusive, required only at commit time to actually finalize the new value).
- **Flow:** writing creates a private uncommitted version without blocking readers of the committed version at all; readers **always** read the committed version (so no dirty reads, no cascading aborts are even possible); at commit, the transaction must acquire a **certify** lock (which *is* exclusive and can force it to wait for existing readers to finish) before the uncommitted version replaces the committed one.

Interview soundbite: "MV2PL's whole trick is separating 'reading' from 'writing' onto two different physical copies, so writers never block readers at all — the only point where exclusivity is truly enforced is the brief certify-lock window right at commit, which is a much smaller contention surface than holding a write lock for a write's entire duration."

7. Validation (Optimistic) Concurrency Control (OCC)

Core idea: don't check for conflicts *during* execution at all — assume there won't be any ("optimistic"), and only actually check right before commit.

Three phases

1. **Read Phase** — reads committed values from the database; all writes go only to a **local, private workspace** — no conflict checking, essentially zero overhead here.

2. **Validation Phase** — before committing, check whether applying this transaction's changes would actually violate serializability, using its recorded **read-set** and **write-set** against other recently committed/currently-validating transactions.
3. **Write Phase** — if validation passes, apply the local workspace's changes to the real database; if it fails, **discard** the workspace and restart the transaction from scratch.

Validation rules (checked in this order, for efficiency)

For transaction **T_i** being validated against another transaction **T_j**, **any one** of these being true is sufficient to be safe:

1. **T_j** completely finishes its write phase **before** **T_i** **even starts** its read phase — no possible overlap at all.
2. **T_i**'s write phase starts **after** **T_j**'s write phase finishes, **and** **T_i**'s read-set doesn't intersect **T_j**'s write-set — **T_i** never read anything **T_j** changed.
3. **T_i**'s read-set and **T_j**'s write-set are disjoint, **and** **T_i**'s and **T_j**'s write-sets are disjoint, **and** **T_j** finishes its read phase before **T_i** finishes its own read phase — they're touching entirely separate data.

If **none** hold, **T_i** is aborted and restarted.

Trade-offs

-  Minimal overhead during actual execution (no locking machinery at all while running); excellent for **low-conflict** workloads (short transactions, rarely-overlapping read/write sets) — genuinely higher concurrency than locking in that regime.
-  In **high-conflict** environments, many transactions repeatedly fail validation and get aborted/restarted — wasted work that locking-based approaches wouldn't have incurred (since locking would have simply made them wait instead of doing the work twice).

Interview soundbite: "OCC is a bet: it bets that conflicts are rare, and pays almost nothing while that bet holds. Locking-based schemes pay a small, steady cost (acquiring/releasing locks) on every transaction regardless of whether a conflict would have happened. Pick OCC when you're confident the bet is good (short transactions, sparse overlap); pick locking when contention is a given."

8. Granularity of Locking

Granularity = the size of the data item being locked — from a single field, up to an entire database.

Granularity	Example	Concurrency	Overhead
Fine (a single record/field)	Lock one row	High — many transactions can proceed in parallel	High — many individual locks to track
Coarse (a whole table/file)	Lock an entire table	Low — even unrelated rows block each other	Low — very few locks to manage

Worked example: T1 wants to modify record r3 ; T2 wants to read r7 , in the same block.

- **Coarse (block-level) locking:** T1 locks the whole block → T2 must wait, even though r3 and r7 don't actually overlap. Simple, but poor concurrency.

- **Fine (record-level) locking:** T1 locks only r3 → T2 can freely proceed on r7 . Better concurrency, but many more individual locks for the system to manage.

Interview soundbite: "Granularity is a direct dial between concurrency and bookkeeping overhead — there's no universally 'correct' choice, which is exactly why real systems support multiple granularity levels simultaneously (row-level, table-level, etc.) and use techniques like intent locks to let a transaction cheaply signal 'I'm about to lock something fine-grained inside this coarse-grained item' without paying the full cost of always locking at the finest level."

Interview Questions With Answers

Q1. Why are binary locks considered impractical despite correctly preventing all conflicts?

Answer: Binary locks allow only one transaction to hold a lock on an item at a time, with no distinction between reading and writing — this means even multiple transactions that only want to *read* the same item (which is always safe to do concurrently, since neither modifies anything) are forced to take turns unnecessarily. This severely limits concurrency for what is typically the most common operation (reads), which is why shared/exclusive locks — which specifically allow concurrent reads — are used in practice instead.

Q2. Explain, using the classic X/Y example, why correctly using locks (acquire before access, release after use) still isn't sufficient to guarantee serializability.

Answer: In the example, T1 reads Y, releases its lock on Y immediately, and only later writes X based on the value it read from Y. Because the lock on Y was released before T1 finished depending on that value, T2 was able to modify Y in the gap — so when T1 finally writes X using its now-stale copy of Y, the result no longer corresponds to any valid serial ordering of T1 and T2. The problem isn't the locking mechanism itself malfunctioning — every lock/unlock operation was individually valid — it's that *releasing a lock too early*, before a transaction is truly done depending on that value, can still let another transaction interfere with an unfinished dependency chain.

Q3. State the two-phase locking rule, and explain why "two-phase" specifically prevents the early-unlock problem.

Answer: 2PL requires a transaction to have two distinct phases: a growing phase where it may only acquire new locks, and a shrinking phase where it may only release locks — once it releases even one lock, it can never acquire another. This directly prevents the early-unlock problem because a transaction can no longer release a lock on an item it might still need to depend on later in its own execution and *then* go acquire a different lock — by the time it starts releasing anything, it must already hold every lock it will ever need, so no other transaction can slip in and invalidate a dependency it's still building on.

Q4. Why can 2PL itself lead to deadlock, given that it solves the serializability problem?

Answer: 2PL requires transactions to hold locks for their entire growing phase (potentially their whole execution) rather than releasing them as soon as they're done with a specific operation — this longer holding time increases the window during which two transactions can each acquire one resource the other needs and then both wait for the other's resource, forming a deadlock cycle. Solving serializability (via strict lock ordering discipline) and avoiding deadlock are genuinely separate concerns, and 2PL's specific solution to the first problem can make the second one more likely, not less.

Q5. Compare Wait-Die and Wound-Wait: how do they differ, and why does each guarantee no deadlock cycle can form?

Answer: Both use each transaction's timestamp (age) to decide who backs off when there's a conflict, but they differ in direction: under Wait-Die, an older transaction requesting an item held by a younger one simply waits, while a younger transaction requesting from an older one aborts and restarts. Under Wound-Wait, it's reversed — an older transaction requesting from a younger one aborts (wounds) the younger holder, while a younger transaction requesting from an older one waits. Each scheme guarantees no cycle can form because only *one specific direction* of the age relationship (older-waits-for-younger under Wait-Die, younger-waits-for-older under Wound-Wait) is ever allowed to result in waiting — a cycle would require waiting to occur in both directions somewhere around the loop, which neither scheme permits by construction.

Q6. What's the difference between deadlock and starvation, and why do Wait-Die/Wound-Wait naturally avoid starvation?

Answer: Deadlock is a cycle of transactions each waiting for a resource held by another, so none can ever proceed — a structural, mutual blocking. Starvation is a single transaction that never gets to make progress, not because of a cycle, but because of unfair scheduling (e.g., it's repeatedly chosen as the abort victim, or newer requests keep cutting in line ahead of it) — there's no cycle involved at all. Wait-Die and Wound-Wait naturally avoid starvation because they base their decisions on a transaction's *fixed* timestamp, which only becomes more favorable relative to newer transactions as

time passes — a transaction can never be perpetually disadvantaged forever, since eventually every other transaction currently running will have a timestamp younger than it.

Q7. What key idea does MVCC add on top of basic timestamp ordering, and what specific benefit does it unlock?

Answer: Basic timestamp ordering maintains only a single current value per data item, and aborts a transaction whenever its read or write would violate the timestamp order relative to that single value. MVCC instead keeps multiple versions of each item, each tagged with its own write and read timestamps. This means a read that would have had to abort under basic timestamp ordering (because the current single value's timestamp bookkeeping conflicts with it) can often instead be served an *older* version of the item that's fully consistent with its own timestamp — turning what would have been an abort into a successful, correct read, at the cost of the extra storage needed to retain multiple versions.

Q8. In MV2PL, why can readers and writers proceed concurrently without blocking each other, and what's the one point where true exclusivity is still enforced?

Answer: MV2PL maintains a committed version (which all readers always read) separate from an uncommitted version (which a writer creates privately while making its changes) — since readers never touch the uncommitted version and writers never touch the committed version until commit time, the two can proceed fully concurrently with no blocking. The one point of true exclusivity is the certify lock, required just before commit to actually replace the committed version with the new value — this lock is exclusive and may force the committing transaction to wait for any transactions still reading the old committed version to finish, but this window is much smaller than holding an exclusive write lock for the writer's entire execution.

Q9. Scenario: Two systems are being designed — one processes short, independent e-commerce order transactions with very low likelihood of touching the same inventory item at the same time; the other processes a high-frequency trading system where many transactions constantly compete to update the same small set of "hot" stock price records. Which concurrency control approach (locking-based like 2PL, or optimistic like OCC) fits each system better, and why?

Answer: For the e-commerce system, OCC is a strong fit — transactions are short, and the described access pattern has genuinely low conflict likelihood (different orders rarely touch the exact same inventory item concurrently), so OCC's near-zero overhead during execution pays off, and the rare validation failure/restart is cheap since so few transactions ever conflict. For the high-frequency trading system, locking-based 2PL is the better fit — the "hot" records being constantly contended by many transactions is exactly the high-conflict scenario where OCC performs poorly, since most transactions would fail validation and have to restart repeatedly, wasting significant work each time; 2PL's approach of simply making transactions wait for a lock (rather than doing full work optimistically and then discovering a conflict at the end) avoids this wasted-work pattern and is the more efficient strategy when conflicts are frequent and expected, not rare exceptions.